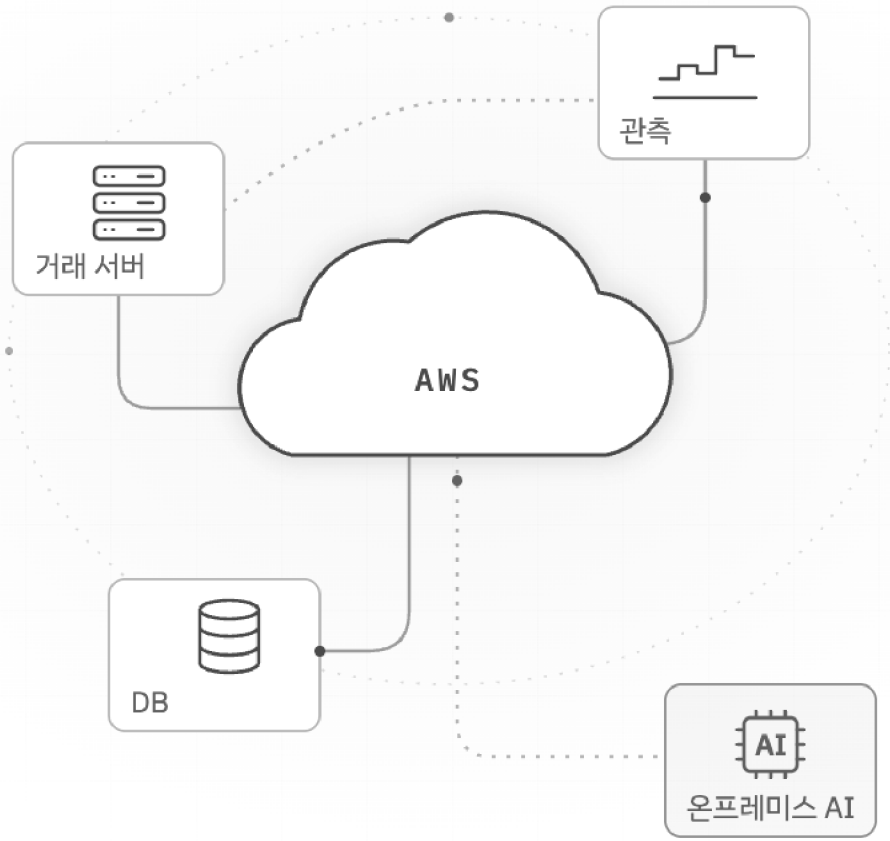
고가용성 플랫폼을 위한 인프라 설계와 AI 운영 자동화



이상혁 | 개인 프로젝트 | 한국폴리텍대학교 광명융합기술교육원 데이터분석과 | 2026.10.16

목차

01	**기획 배경**	고가용성을 보여 주려고 거래소를 고른 이유
02	**기획 시나리오**	요청이 몰릴 때와 DB가 멈출 때의 두 장면
03	**설계 방향**	두 장면에 답하도록 그린 전체 구조
04	**주요 설계 및 구현**	Docker부터 VM·Linux·AWS를 거쳐 AI 관제까지
05	**시연**	부하 증설과 DB 자동 전환을 담은 7분 영상
06	**마무리**	시험으로 확인한 결과와 앞으로의 다짐

기획 배경

월 30만 원으로 구축하는 고가용성 인프라 프로젝트

제약 조건 운영 예산 **30만 원** 운영 인력 **1명** 배포 기간 **한 달**

통장 잔액

300,000원



이번 인프라 프로젝트, 뭘로 하지?

내 인프라 역량을 총동원해서, **고가용성 플랫폼**을 만들자.

대규모 트래픽을 견디고, **중요한 데이터**를 지키고,
장애 상황도 버틸 수 있어야 해.

아하! 그렇다면, **가상화폐 거래소**를 만들어 보자.

근데... 통장에 **30만 원**밖에 없네?

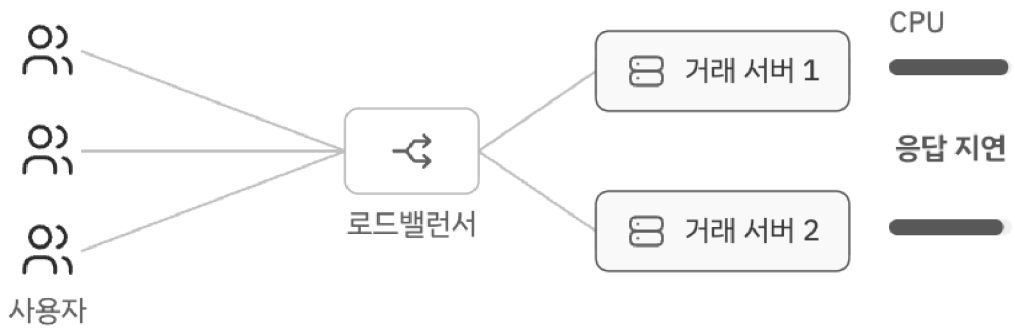
운영도 혼자 해야 하니, **관제에 AI**를 활용하자.

기획 시나리오

거래소를 실제로 운영한다면 가장 먼저 겪게 될 두 장면을 골랐습니다.

01 트래픽 급증

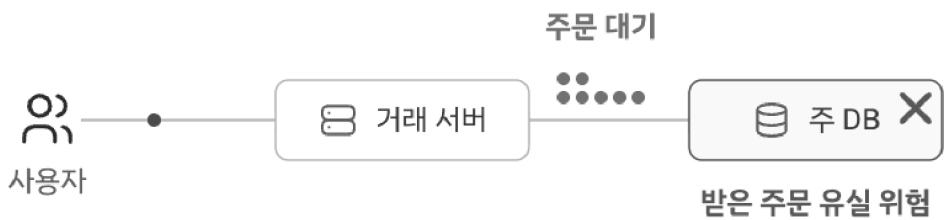
요청이 한꺼번에 몰려도 서버는 버텨야 한다



확인할 것 | 어디서 먼저 막히는지, 무엇을 늘려야 하는지

02 DB 장애

DB가 갑자기 멈춰도 받은 주문은 지켜야 한다



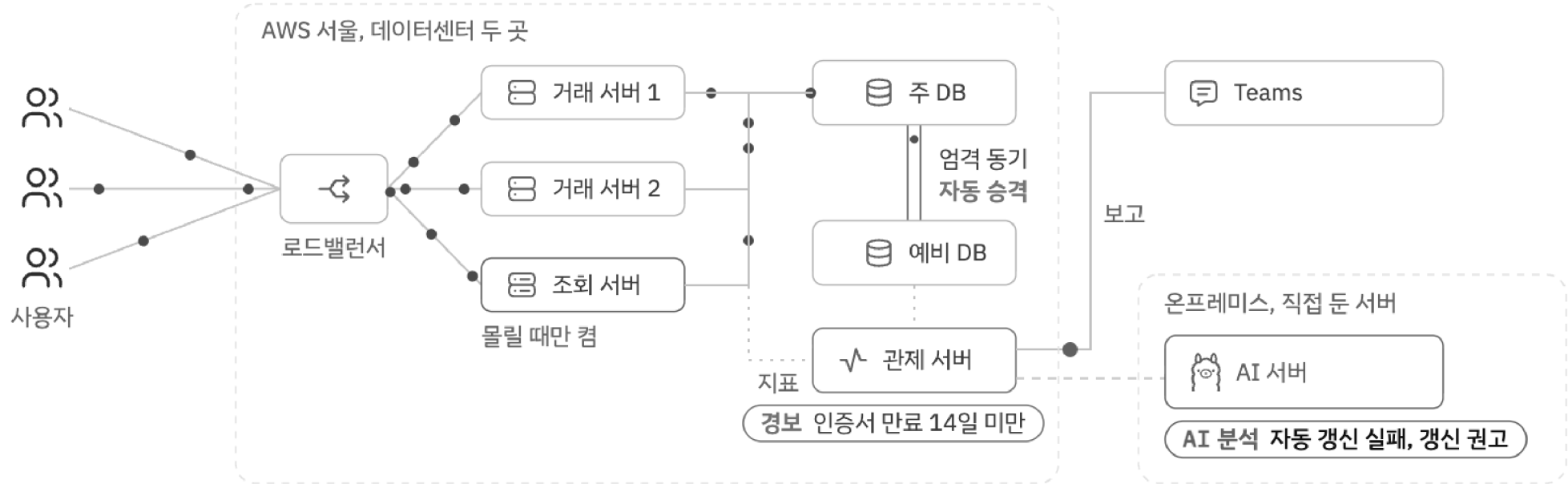
확인할 것 | 받은 주문이 남는지, 사람 없이 넘어가는지

AI의 역할 흩어진 경보와 지표를 모아 원인을 짚고, 다음에 할 일을 제안합니다.

설계 방향

월 30만 원으로 혼자 한 달 안에 두 장면에 답하도록 설계했습니다.

`01 + 02` **계획한 아키텍처**



주요 설계 및 구현 순서

로컬 Docker에서 먼저 시험한 뒤 VM을 거쳐 AWS에 배포했습니다.

● AWS 배포 완료

01

Docker 실험

로컬 Docker

만든 것

PC 한 대에 띄운 실제와 같은 구성

확인한 것

장애와 부하를 재현해 잰 수치

02

망 설계

Packet Tracer

만든 것

역할별로 망을 나눈 논리 설계

정한 것

필요한 길만 여는 허용·차단 규칙

03

Linux VM

VM 실습망

만든 것

투명 방화벽·RAID·이중화·백업

확인한 것

차단·디스크 교체·백업 복원 시험

04

AWS 배포

학교 AWS → 개인 AWS

만든 것

역할별 서버와 DB 자동 전환

확인한 것

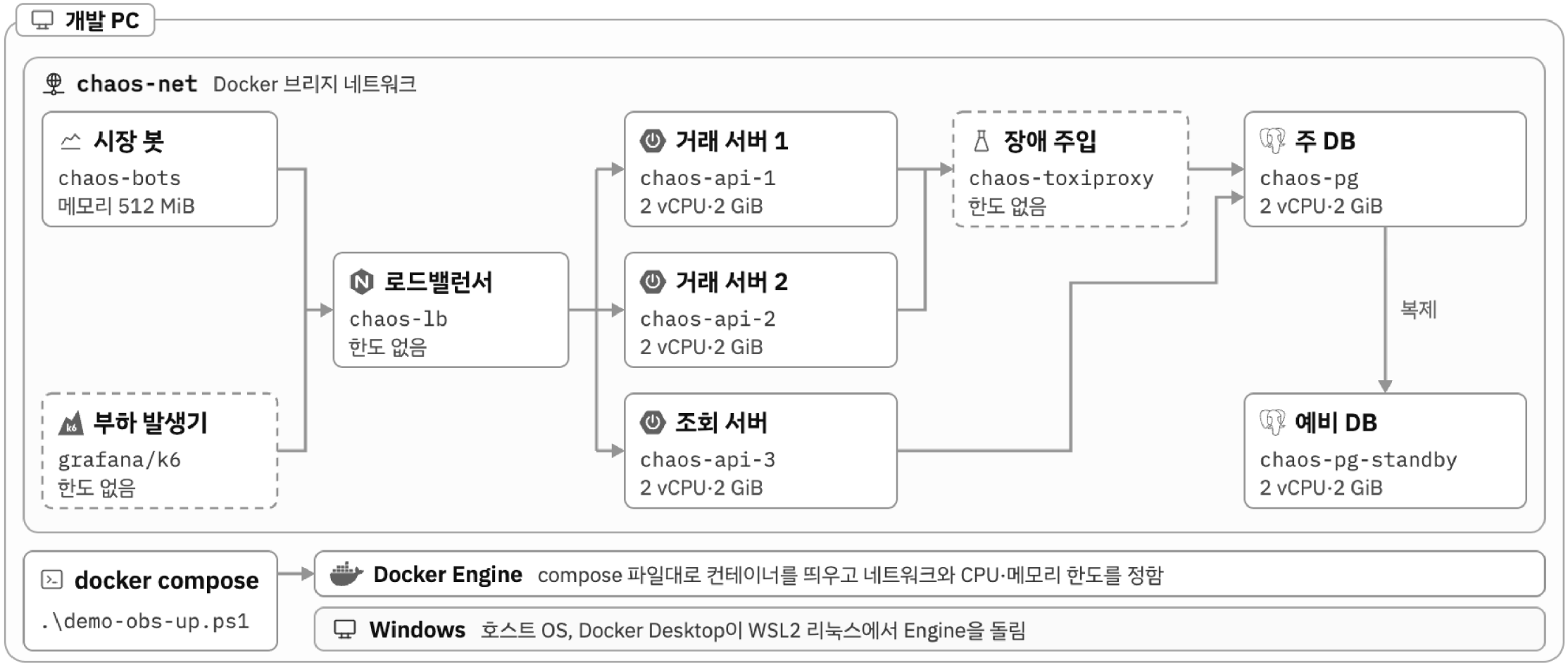
학교 EC2에서 리눅스 운영 점검

뒤 장의 수치는 이 네 단계에서 잰 값이고, 어디서 쟀는지는 장마다 오른쪽 위에 로컬 Docker 같은 표시로 밝혔습니다.

Docker 시험 환경

로컬 Docker

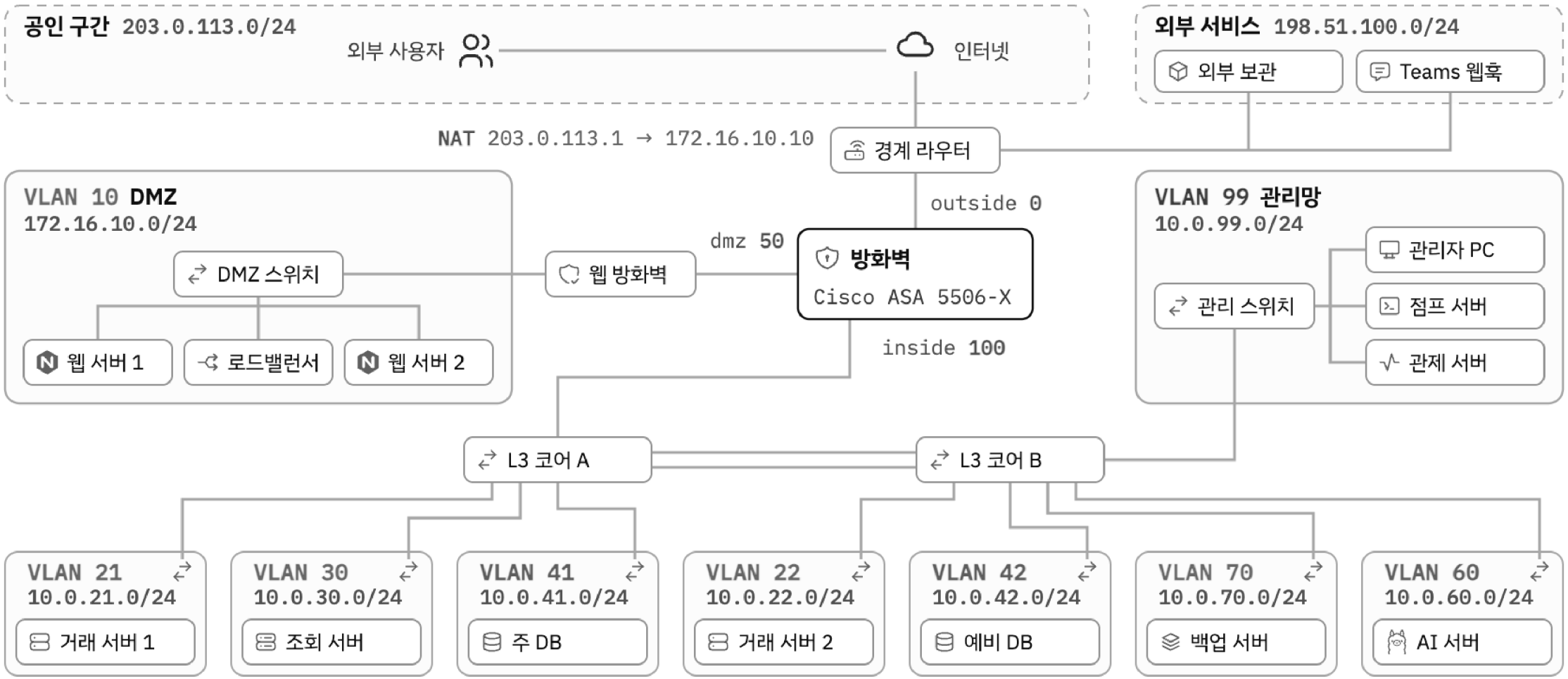
API·DB 컨테이너는 실습 VM 서버와 같은 **2 vCPU·2 GiB**로 제한했습니다.



네트워크 구성도

Packet Tracer

역할마다 VLAN을 나누고 그 사이에는 필요한 통신만 열었습니다.



VM 구성도

VM 실습망

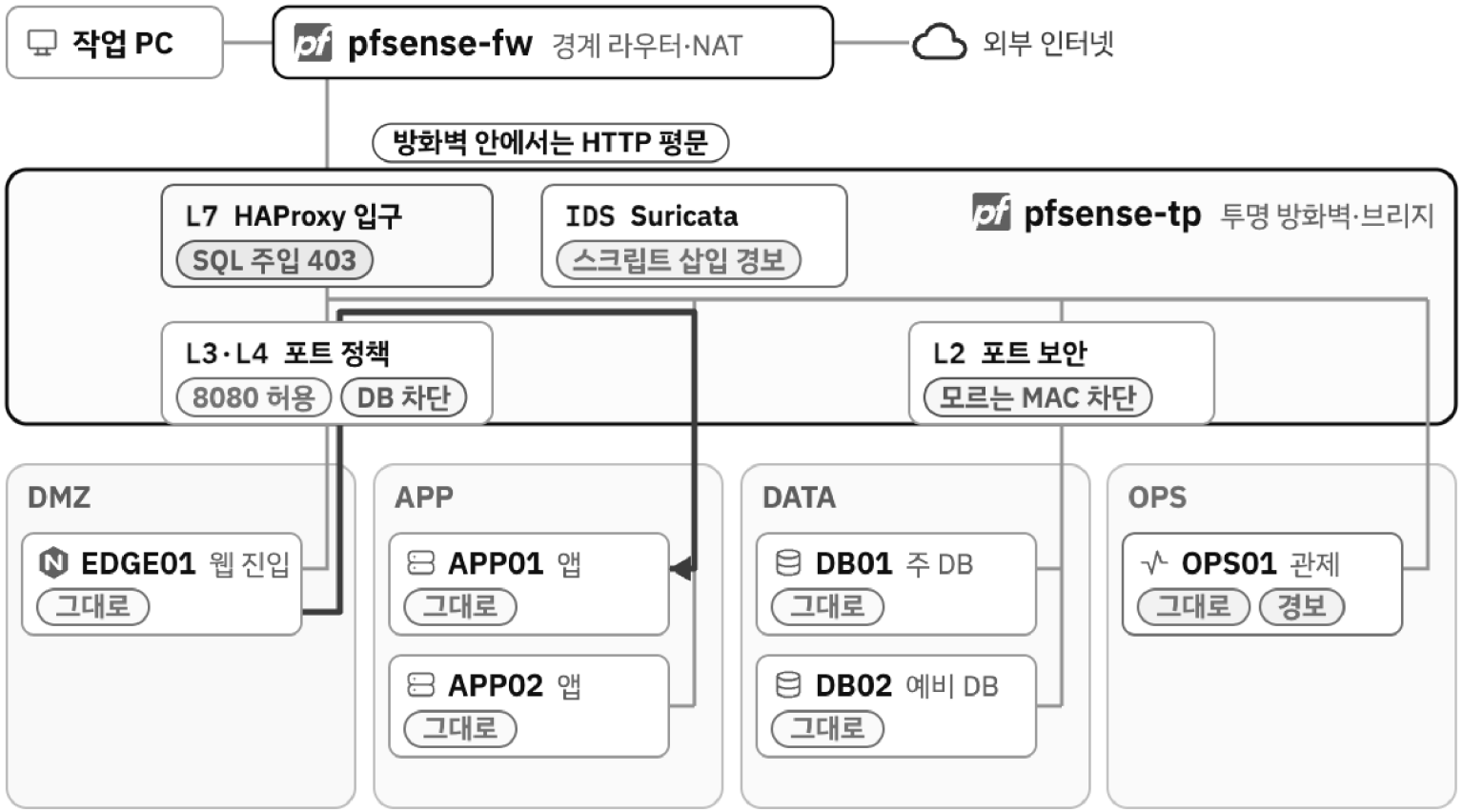
pfSense 라우터 뒤 한 대역에 서버 여섯 대를 두고 시작했습니다.



HAProxy와 Suricata

VM 실습망

서버 주소를 바꾸지 않는 투명 브리지에서 요청 내용까지 검사했습니다.



방화벽을 끼울 때 서버 쪽 변경

변경 **경계형**
게이트웨이·DNS·NTP를 192.168.10.1로

그대로 **투명 브리지**
게이트웨이·DNS·NTP 그대로

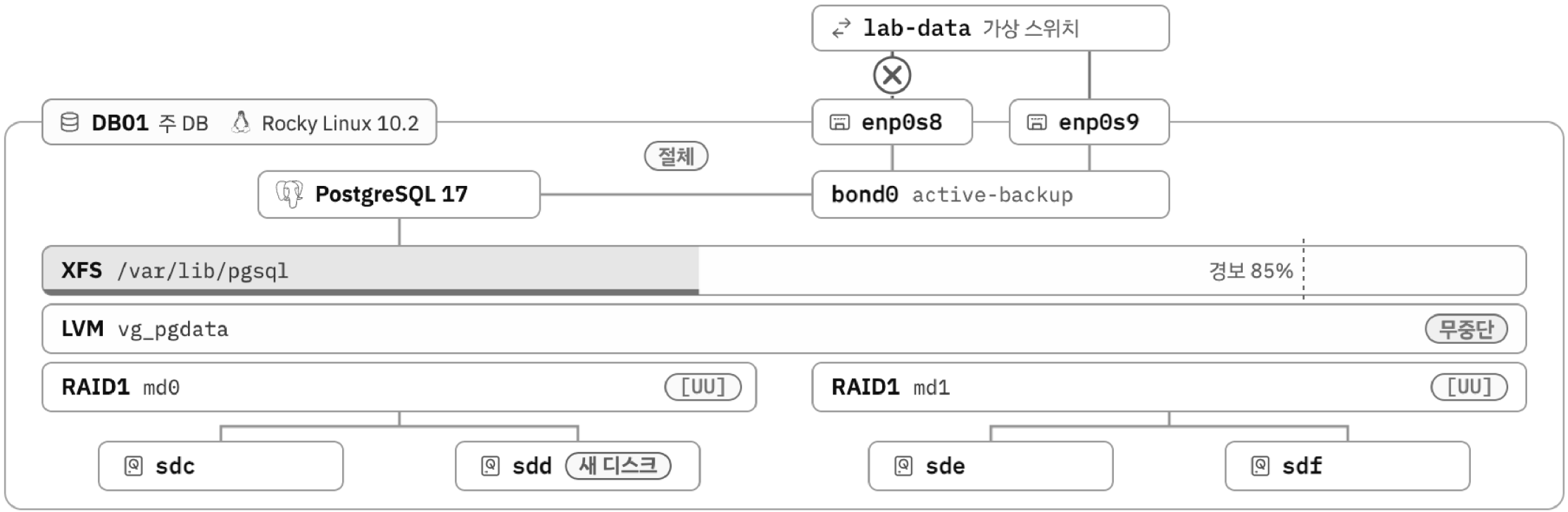
입구에서 보이는 정보

L2	em0 포트, 라우터 MAC
L3	작업 PC → 입구 192.168.10.10
L4	TCP 443, TLS 1.3
L7	GET /api/v1/markets

RAID 구성과 네트워크 이중화

VM 실습망

주 DB 서버에 RAID1·LVM·bonding을 직접 구성하고, 디스크를 빼고 선을 끊어 동작을 확인했습니다.



경보 ● **네트워크 링크 장애 발생 · bond0 링크 하나 끊김**

커널 자동 `bond0: (slave enp0s9): making interface the new active one`

결과 `Currently Active Slave: enp0s9`

DB 쓰기 검사

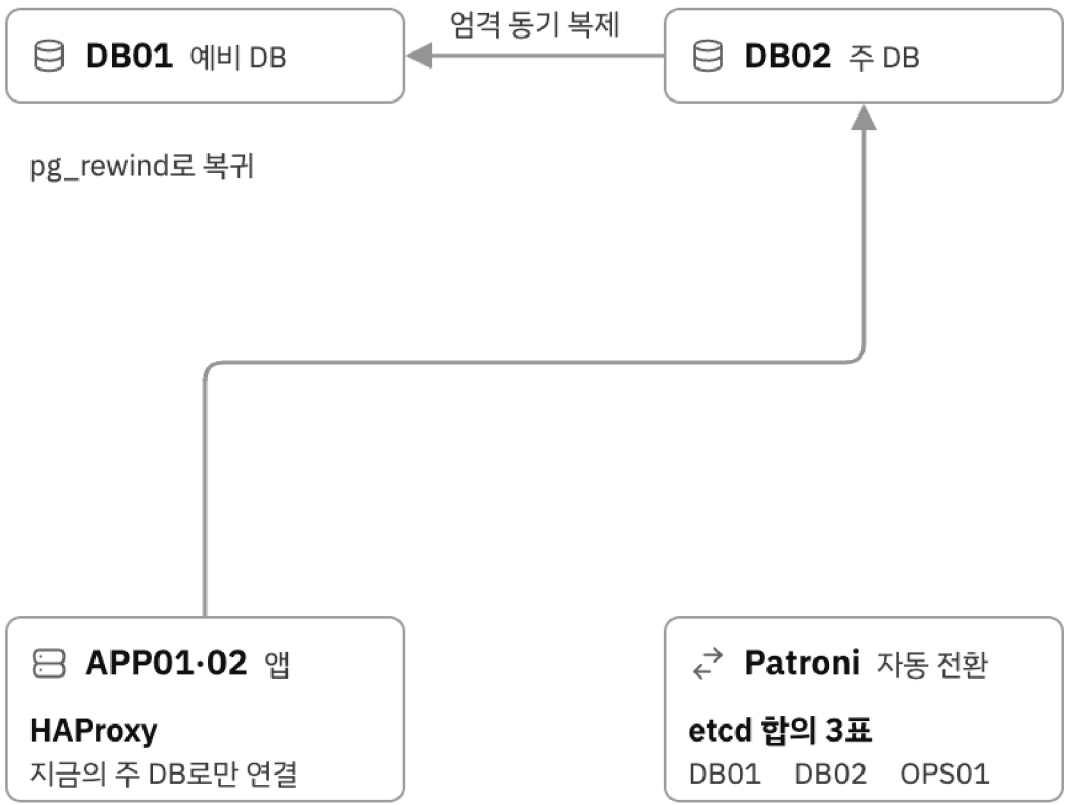
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

DB 이중화와 백업

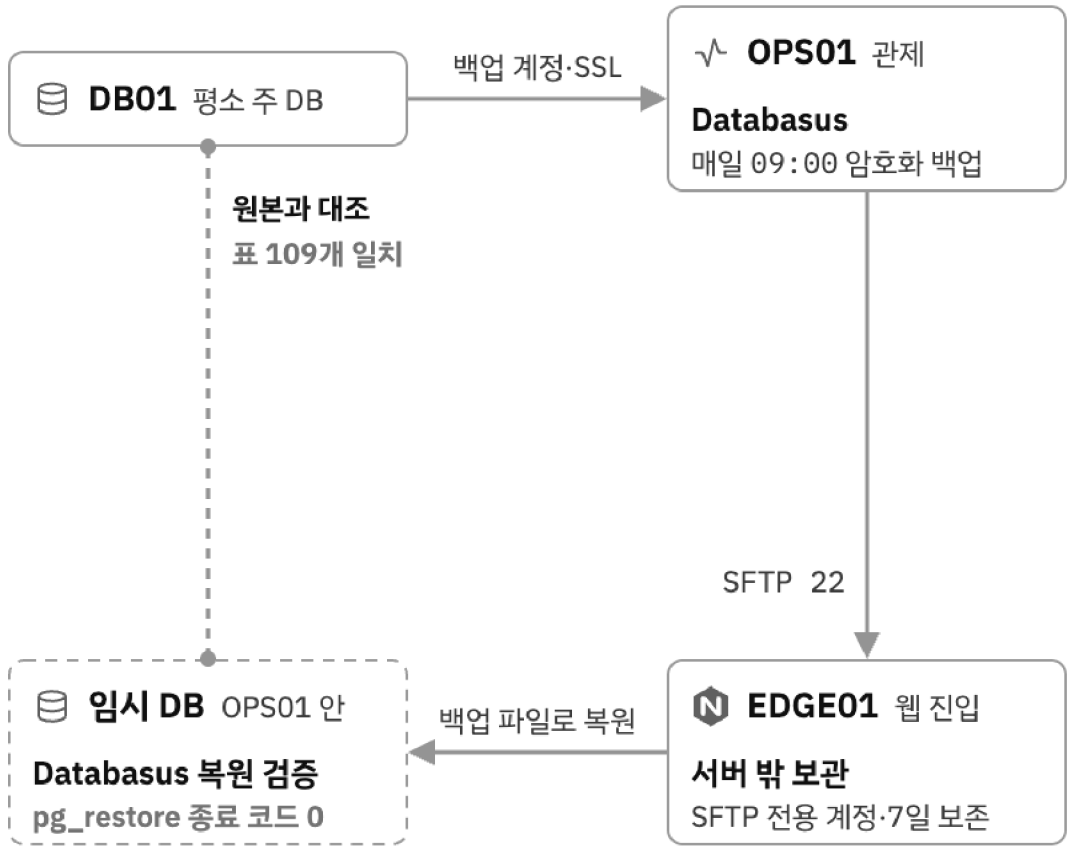
VM 실습망

복제만으로는 잘못 지운 데이터를 되돌릴 수 없어 백업을 두고 복원까지 해 봤습니다.

❶ 이중화 서버가 멈출 때



❷ 백업 지우거나 망가졌을 때



리눅스 서버 운영

학교 AWS

학교 EC2에 서비스를 직접 설치하고 설정마다 명령 출력으로 확인했습니다.

- **계정·권한** 서비스 전용 계정
- **서비스** systemd
- **로그** journald
- **네트워크** Nginx·HTTPS
- **디스크** 용량 경보

직접 설치한 구성 요소 모두 systemd 서비스로 운영

`app-1`	거래 서버	Java 21 앱, Nginx, Certbot
`db-primary·db-standby`	주 DB·예비 DB	PostgreSQL 17
`bot-1`	관제 서버	Prometheus, Grafana, Alertmanager, 시장 봇

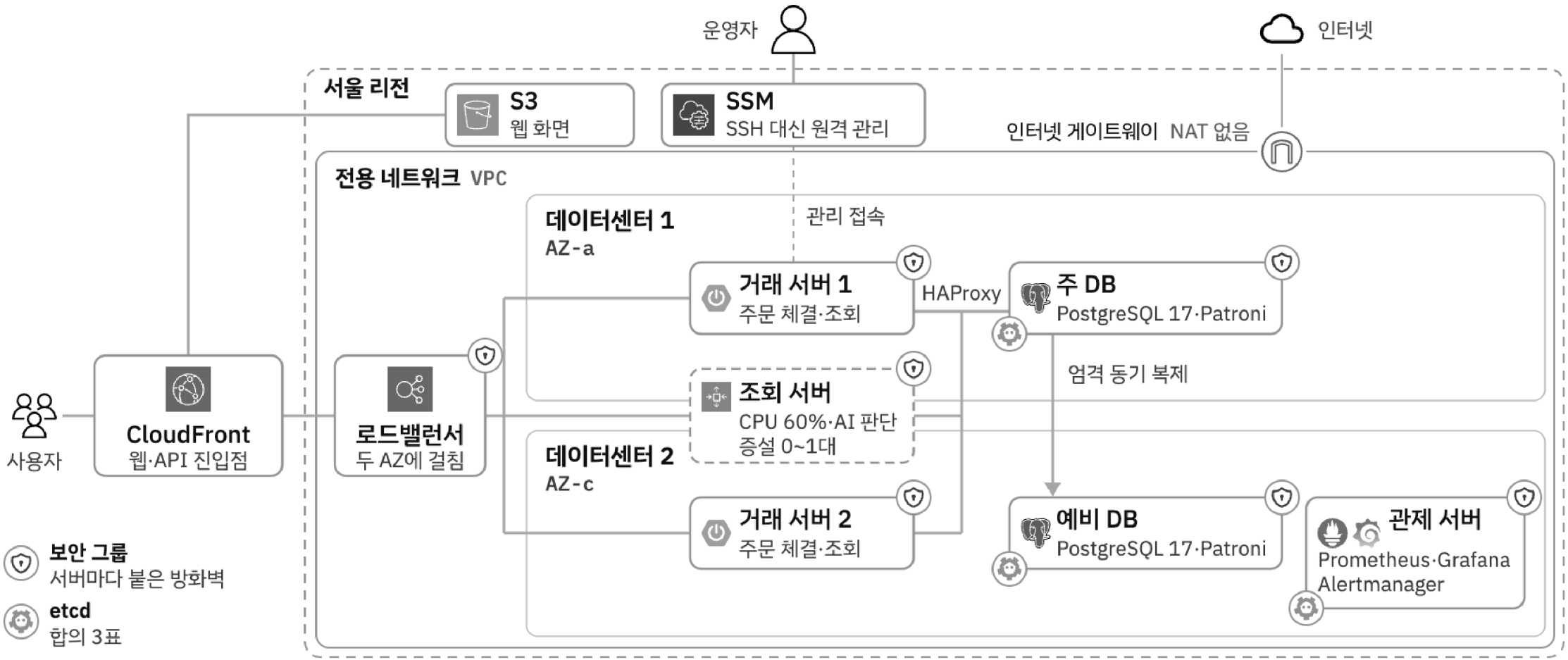
읽는 법 분야마다 같은 순서

1. **설정** 직접 넣은 설정
2. **시험** 일부러 일으킨 상황
3. **결과** 명령 출력으로 확인

AWS 구성도

AWS 배포 구성

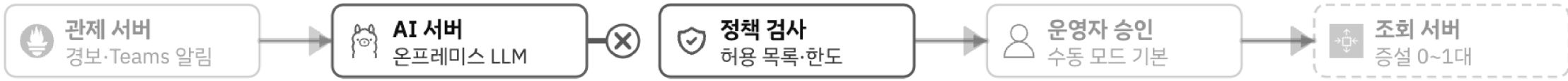
개인 AWS의 두 AZ에 서버를 나누고 들어오는 길은 CloudFront와 SSM으로 좁혔습니다.



AI 관제

관제센터 시나리오 재생

AI가 낸 조치안은 정책 검사를 거치고 운영자가 승인해야 실행됩니다.



0 /111
잘못 실행될 조치

9 /9
조건이 안 맞는 제안을 정책 검사가 막음

12 /12
명령을 섞은 경보에 따르지 않음

37 /37
같은 경보를 세 번 넣어도 같은 판단

판단 평가

qwen3.6 35B·111회
경보 상황마다 판단만 시키고
자동 모드였다면 실행됐을 조치를 셈

차단 조건 밖·금지 요청은 정책 검사가 막음

시연 영상

개발 PC, 로컬 Docker ▶ 영상 재생 7:00

배포가 전부 끝나고 녹화한 7분 영상입니다.



01 0:00–1:13

거래와 평상시 관제

- 시장 봇이 주문하는 리플 1분봉 화면
- 수동 모드로 둔 AI 관제센터
- 2초마다 실측값으로 그린 서비스 흐름
- Grafana 서비스 관제

보여 줄 것

관제 화면의 숫자는 모두
실제 거래에서 잰 값입니다.

02 1:13–4:14

부하 시험과 AI 증설

- 부하 발생기로 **초당 9,000건**
- 앱 CPU가 증설 기준을 넘어 경보
- AI가 조회 서버 켜기를 제안
- 승인 뒤 앱 서버 3이 조회 서버로 합류

보여 줄 것

운영자가 승인한 뒤에
조회 서버가 켜집니다.

03 4:14–6:12

DB 장애와 자동 전환

- 주 DB 강제 종료로 끊긴 서비스 흐름
- Patroni가 **27초** 만에 예비 DB 승격
- 옛 주 DB 되감기 합류 뒤 거래 재개
- 확인받은 쓰기 대조로 사라진 주문 **0건**

보여 줄 것

거래는 잠시 멈추지만
확인받은 주문은 남습니다.

04 6:12–7:00

AI 상담과 요약

- AI에게 방금 일을 질문
- 조회 전용이라 실행은 없음
- 답에 근거·모델·수집 시각
- 실측 요약 화면으로 마무리

보여 줄 것

AI는 답만 하고
명령은 실행하지 않습니다.

연결 앞에서 설계한 구성으로 기획한 두 장면을 실제로 돌렸고, 증설 제안은 앞 장 AI 관제와 같은 흐름입니다.

장애 보고 전달

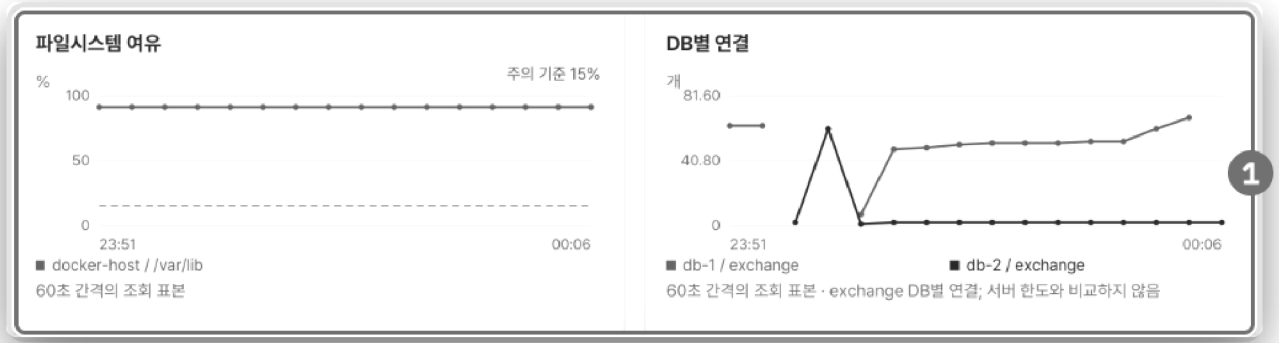
개발 PC, 로컬 Docker

경보가 나면 무엇을 확인해야 하는지 Teams 카드와 2쪽 보고서로 알립니다.

Teams 카드
담당자에게 가는 한 장

보고서 1쪽
우선순위·자동 조치·종결 기준

보고서 2쪽
지표 추이·확인 못 한 자료



1

미확인 자료

수신 네트워크 오류: 자료 없음; 실패 서비스: 자료 없음; 호스트 운영 기록: 자료 없음; 상세 로그: 자료 없음; AWS 변경 기록: 자료 없음; 백업 예약 확인: 자료 없음 · 외 1항목: HTML 상세 보고 참조

2

1 지표 추이
근거 수집 전 15분을 60초 간격으로 기준선과 함께 그림

2 확인 못 한 자료
백업 예약 확인처럼 값이 없으면 0이 아니라 '자료 없음'으로 적음

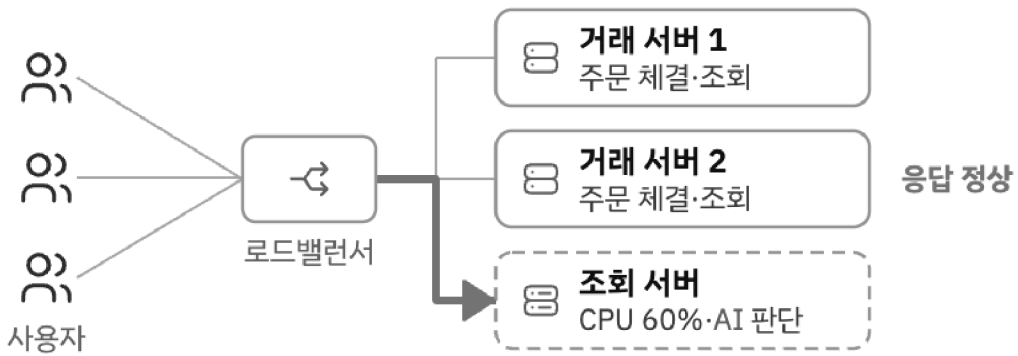
작성 방식 AI 분석은 참고로만 붙임

마무리

기획 시나리오의 두 장면을 그대로 재현해 시험했습니다.

01 트래픽 급증 로컬 Docker

조회 서버를 켜서 몰린 요청을 나눠 받는다

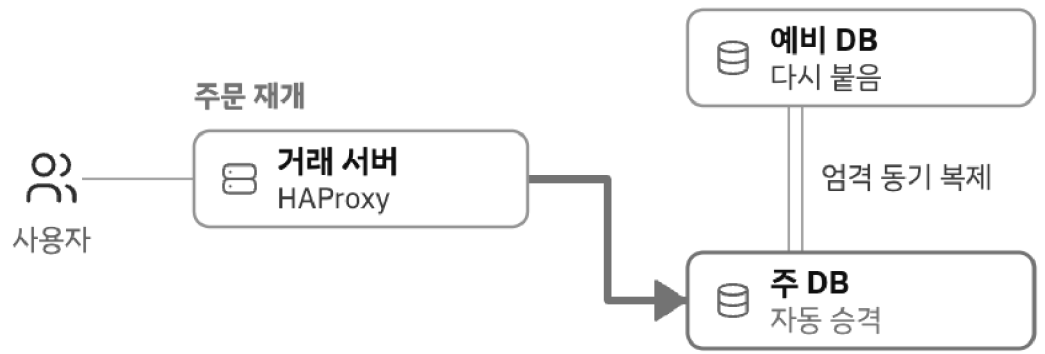


9,000건
초당 몰린 요청

18초
승인 뒤 조회 서버 합류

02 DB 장애 로컬 Docker

예비 DB가 스스로 이어받고 받은 주문은 지킨다



27초
예비 DB가 주 DB로 오르기까지

0건
확인받은 쓰기 16,858건 중 유실

두 장면을 시험하는 동안 받은 주문은 한 건도 잃지 않았습니다
고객사 서버에서도 직접 확인한 것만 보고하는 엔지니어가 되겠습니다

감사합니다